

Dateiauswertung

GLEICH

.git/hooks/pre-rebase.sample

Branch 1	main
Branch 2	current
Datei	.git/hooks/pre-rebase.sample
Typ	Datei

Bewertung

SHA-256 identisch.

Dateiinhalt / Vergleich

1	#!/bin/sh
2	#
3	# Copyright (c) 2006, 2008 Junio C Hamano
4	#
5	# The "pre-rebase" hook is run just before "git rebase" starts doing
6	# its job, and can prevent the command from running by exiting with
7	# non-zero status.
8	#
9	# The hook is called with the following parameters:
10	#
11	# \$1 -- the upstream the series was forked from.
12	# \$2 -- the branch being rebased (or empty when rebasing the current branch).
13	#
14	# This sample shows how to prevent topic branches that are already
15	# merged to 'next' branch from getting rebased, because allowing it
16	# would result in rebasing already published history.
17	
18	publish=next
19	basebranch="\$1"
20	if test "\$#" = 2
21	then
22	topic="refs/heads/\$2"
23	else
24	topic=`git symbolic-ref HEAD`
25	exit 0 ;# we do not interrupt rebasing detached HEAD
26	fi
27	
28	case "\$topic" in
29	refs/heads/??/*)
30	;;

31	*)
32	exit 0 ;# we do not interrupt others.
33	;;
34	esac
35	
36	# Now we are dealing with a topic branch being rebased
37	# on top of master. Is it OK to rebase it?
38	
39	# Does the topic really exist?
40	git show-ref -q "\$topic" {
41	echo >&2 "No such branch \$topic"
42	exit 1
43	}
44	
45	# Is topic fully merged to master?
46	not_in_master=`git rev-list --pretty=oneline ^master "\$topic"`
47	if test -z "\$not_in_master"
48	then
49	echo >&2 "\$topic is fully merged to master; better remove it."
50	exit 1 ;# we could allow it, but there is no point.
51	fi
52	
53	# Is topic ever merged to next? If so you should not be rebasing it.
54	only_next_1=`git rev-list ^master "^\$topic" \${publish}   sort`
55	only_next_2=`git rev-list ^master \${publish}   sort`
56	if test "\$only_next_1" = "\$only_next_2"
57	then
58	not_in_topic=`git rev-list "^\$topic" master`
59	if test -z "\$not_in_topic"
60	then
61	echo >&2 "\$topic is already up to date with master"
62	exit 1 ;# we could allow it, but there is no point.
63	else
64	exit 0
65	fi
66	else
67	not_in_next=`git rev-list --pretty=oneline ^\${publish} "\$topic"`
68	/usr/bin/perl -e '
69	my \$topic = \$ARGV[0];
70	my \$msg = "* \$topic has commits already merged to public branch:\n";
71	my (%not_in_next) = map {
72	/^[0-9a-f]+) /;
73	(\$1 => 1);
74	} split(/\n/, \$ARGV[1]);
75	for my \$elem (map {
76	/^[0-9a-f]+) (.*)\$/;
77	[\$1 => \$2];
78	} split(/\n/, \$ARGV[2])) {
79	if (!exists \$not_in_next{\$elem->[0]}) {
80	if (\$msg) {
81	print STDERR \$msg;

82	undef \$msg;
83	}
84	print STDERR " \$elem->[1]\n";
85	}
86	}
87	' "\$topic" "\$not_in_next" "\$not_in_master"
88	exit 1
89	fi
90	
91	<<\DOC_END
92	
93	This sample hook safeguards topic branches that have been
94	published from being rewound.
95	
96	The workflow assumed here is:
97	
98	* Once a topic branch forks from "master", "master" is never
99	merged into it again (either directly or indirectly).
100	
101	* Once a topic branch is fully cooked and merged into "master",
102	it is deleted. If you need to build on top of it to correct
103	earlier mistakes, a new topic branch is created by forking at
104	the tip of the "master". This is not strictly necessary, but
105	it makes it easier to keep your history simple.
106	
107	* Whenever you need to test or publish your changes to topic
108	branches, merge them into "next" branch.
109	
110	The script, being an example, hardcodes the publish branch name
111	to be "next", but it is trivial to make it configurable via
112	\$GIT_DIR/config mechanism.
113	
114	With this workflow, you would want to know:
115	
116	(1) ... if a topic branch has ever been merged to "next". Young
117	topic branches can have stupid mistakes you would rather
118	clean up before publishing, and things that have not been
119	merged into other branches can be easily rebased without
120	affecting other people. But once it is published, you would
121	not want to rewind it.
122	
123	(2) ... if a topic branch has been fully merged to "master".
124	Then you can delete it. More importantly, you should not
125	build on top of it -- other people may already want to
126	change things related to the topic as patches against your
127	"master", so if you need further changes, it is better to
128	fork the topic (perhaps with the same name) afresh from the
129	tip of "master".
130	
131	Let's look at this example:
132	

133	o---o---o---o---o---o---o---o---o "next"
134	/ / / /
135	/ a---a---b A / /
136	/ / / /
137	/ / c---c---c---c B /
138	/ / / \ /
139	/ / / b---b C \ /
140	/ / / / \ /
141	---o---o---o---o---o---o---o---o---o "master"
142	
143	
144	A, B and C are topic branches.
145	
146	* A has one fix since it was merged up to "next".
147	
148	* B has finished. It has been fully merged up to "master" and "next",
149	and is ready to be deleted.
150	
151	* C has not merged to "next" at all.
152	
153	We would want to allow C to be rebased, refuse A, and encourage
154	B to be deleted.
155	
156	To compute (1):
157	
158	git rev-list ^master ^topic next
159	git rev-list ^master next
160	
161	if these match, topic has not merged in next at all.
162	
163	To compute (2):
164	
165	git rev-list master..topic
166	
167	if this is empty, it is fully merged to "master".
168	
169	DOC_END